

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ
по ознакомительной практике

Выполнил:

М. В. Вашкевич

Студент группы
421702

Проверил:

Н. В. Малиновская

Минск 2025

СОДЕРЖАНИЕ

Введение	3
1 Формализация и сравнительный анализ языка описания онтологий СусL и функционального языка программирования LISP	4
2 Формализация и сравнительный анализ инструментов для разработ- ки интеллектуальных систем: NELL и XLore	9
Заключение	14
Список использованных источников	16

ВВЕДЕНИЕ

Цель:

Провести сравнительный анализ современных технологий разработки интеллектуальных систем, закрепить навыки формализации информации, а также приобрести и развить навыки исследования, анализа и сравнения технологических решений.

Задачи:

1. Изучить язык описания онтологий CusL и функциональный язык программирования LISP как инструменты формализации декларативных знаний, провести анализ их особенностей, выявить сходства и различия между ними.
2. Рассмотреть современные инструменты для работы с графами и базами знаний (NELL и XLORE), выявить их функциональные возможности и подходы к представлению знаний, сходства и отличительные особенности.
3. Формализовать полученные результаты в виде SCn-кода.
4. Оформить библиографические источники к полученным результатам по ГОСТ 7.1-2003.

1 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЯЗЫКА ОПИСАНИЯ ОНТОЛОГИЙ CYCL И ФУНКЦИОНАЛЬНОГО ЯЗЫКА ПРОГРАММИРОВАНИЯ LISP

CycL

- :=** [Язык описания онтологий, используемый в проекте искусственного интеллекта Cyc]
- :=** [CycL (Cyc Language)]
- ∈** *язык описания онтологий*
- ⇒** *разработчик**:
 - *Дуглас Ленат*
 - *Раманатан В. Гуха*
- ⇒** *год создания**:
1984
- ⇒** *основная парадигма**:
декларативное программирование
- ⇒** *описание**:

[CycL — формальный язык представления знаний, разработанный для проекта Cyc, одной из самых масштабных в мире баз знаний, целью которой является кодирование здравого смысла и сложных концепций через формализацию общих знаний. Программы на CycL состоят из утверждений о мире, включающих аксиомы, правила вывода и объекты, а выполнение осуществляется через алгоритмы автоматического доказательства теорем, учитывающие семантическую связанность и иерархию понятий.]
- ⇒** *описание**:

[CycL поддерживает формализацию отношений между объектами через аксиомы, правила и утверждения, позволяя строить глубокие логические выводы. Обеспечивает динамическое обновление баз знаний. Язык активно используется для создания экспертных систем, анализа данных и решения задач, требующих контекстного понимания.]
- ⇒** *основные конструкции**:
 - { • [факт]
 - [правило]
 - [микротеория]
 - }
- ⇒** *стратегия вывода**:
Логический вывод с учетом контекста
- ⇒** *поддержка**:
 - [контекстное программирование]
 - [онтологическая модульность]
 - [встроенные правила наследования]
- ⇒** *применение**:
 - *искусственный интеллект*
 - *представление знаний*
 - *реализация систем, обладающих здравым смыслом*
 - *обработка естественного языка*
 - *машинное обучение*
 - *сложный логический вывод*
- ⇒** *примеры использования**:
 - { •

- [Создание баз знаний, связывающих симптомы, заболевания и методы лечения, с возможностью логического вывода на основе здравого смысла]
 - [Реализация чат-ботов или виртуальных помощников, способных понимать контекст запросов и давать логически обоснованные ответы]
 - [Анализ тональности и выявление скрытых смыслов в документах]
 - [Аннотирование данных, что улучшает обучение моделей в задачах вроде распознавания образов или прогнозирования поведения пользователей]
 - [Создание интеллектуальных тренажёров, которые анализируют ошибки ученика и предлагают персонализированные пути обучения, опираясь на здравый смысл]
- }
⇒ *реализации**:
{• *Cyc Core*
• *OpenCyc*
}
⇒ *особенности**:
{• [строгая типизация через микротеоии]
• [высокая выразительность]
• [интеграция с онтологиями]
• [поддержка неполных знаний]
}
⇒ *ограничения**:
{• [специфичность для системы Cyc]
• [ограниченная гибкость вне контекста онтологий]
}
⇒ *библиографические источники**:
{• [Lenat D. B., Guha R. V. Building Large Knowledge-Based Systems. Addison-Wesley, 1990.]
• [Lenat, D.B. The Dimensions of Context Space / D.B. Lenat, R.V. Guha. — Stanford, CA: Stanford University, 1989. — 152 p.]
• [Matuszek, C. Using Cyc to Enable Natural Language Interaction with a Commercial Game / C. Matuszek [и др.] // Proceedings of the 20th International Florida Artificial Intelligence Research Society Conference (FLAIRS-20). — Menlo Park, CA: AAAI Press, 2007. — P. 456–461.]
• [CycL // Wikipedia – 2025. – Электронный ресурс. Дата обращения: 24.05.2025. <https://en.wikipedia.org/wiki/CycL>]
• [Документация Cyc – 2025. – Электронный ресурс. Дата обращения: 25.05.2025. <https://www.cyc.com>]
}

LISP

- := [Функциональный язык программирования, разработанный Джоном Маккарти]
:= [LISP (LISt Processing language)]
∈ *функциональный язык*
⇒ *разработчик**:
• *Джон Маккарти*
⇒ *год создания**:
1958
⇒ *основная парадигма**:
функциональное программирование

- ⇒ *описание**:
- [LISP — универсальный язык функционального программирования для работы с символьными данными и списковыми структурами. Язык основан на концепции S-выражений, где код и данные представлены в единой форме, что позволяет использовать программы как данные для метапрограммирования через макросы. LISP ориентирован на обработку иерархических данных, рекурсивных вычислений и динамических структур.]
- ⇒ *описание**:
- [LISP оптимален для работы с высокоабстрактными структурами данных, реализации сложных алгоритмов искусственного интеллекта и метапрограммирования. Благодаря своей гибкости, основанной на унифицированном представлении кода и данных через списки, а также мощной системе макросов, он обеспечивает высокую выразительность и адаптивность.]
- ⇒ *назначение**:
- [LISP обеспечивает гибкость и выразительность при разработке программ, особенно в областях, где требуется манипуляция списками, символьными выражениями и динамическая генерация кода.]
- ⇒ *основные конструкции**:
- { • [S-выражение]
 - [функция]
 - [макрос]
 - }
- ⇒ *стратегия вывода**:
- Интерпретация и компиляция функциональных выражений*
- ⇒ *поддержка**:
- [рекурсия]
 - [динамическое управление памятью]
 - [макросы для расширения языка]
- ⇒ *применение**:
- *искусственный интеллект*
 - *обработка естественного языка*
 - *математические вычисления*
 - *формальные доказательства*
 - *прототипирование алгоритмов*
- ⇒ *примеры использования**:
- { • [Парсеры для разбора грамматических конструкций или системы машинного перевода]
 - [Реализация систем, упрощающих математические выражения или находящих аналитические решения уравнений]
 - [Эксперименты с новыми алгоритмами, такими как генетические программирование или нейронные сети]
 - [Реализация алгоритмов для задач, требующих анализа сложных структур данных]
 - }
- ⇒ *реализации**:
- { • *Allegro Common Lisp*
 - *CLISP*
 - *Clozure CL*
 - *SBCL (Steel Bank Common Lisp)*
 - *CMUCL*

- *ECL (Embeddable Common Lisp)*
- }
- ⇒ *особенности**:
 - {• [минималистичный синтаксис]
 - [гибкость метапрограммирования]
 - [формализация онтологии функций]
 - }
- ⇒ *ограничения**:
 - {• [ограничение алфавита и регистронезависимость]
 - [проблемы интеграции]
 - [специфичность применения]
 - }
- ⇒ *библиографические источники**:
 - {• [LISP | Artificial Intelligence, Machine Learning & Programming // Massachusetts Institute of Technology (MIT). – Электронный ресурс. Дата обращения: 25.05.2025. <https://www.britannica.com/technology/LISP-computer-language>]
 - [Лисп // Википедия – 2025. – Электронный ресурс. Дата обращения: 25.05.2025. <https://ru.wikipedia.org/wiki/Лисп>]
 - [What is the Lisp (List Processing) Programming Language? // TechTarget. – 2022. – Электронный ресурс. Дата обращения: 25.05.2025. <https://www.techtarget.com/whatis/definition/LISP-list-processing>]
 - [Введение в программирование систем искусственного интеллекта [PDF] // Джон Маккарти. – 1958.]
 - [Введение в язык Лисп и функциональное программирование. // Хабр. – 2023. – Электронный ресурс. Дата обращения: 25.05.2025. <https://habr.com/ru/articles/764594/>]
 - }

Сравнение языка описания онтологий CysL и языка функционального программирования LISP

- = {• *CysL*
- *LISP*
- }
- ⇒ *сходства**:
 - {• [Оба применяются в задачах искусственного интеллекта и обработки знаний.]
 - [Поддерживают формализацию сложных структур данных.]
 - [Имеют открытые и коммерческие реализации, используются в научных и промышленных проектах]
 - }
- ⇒ *различия**:
 - {• [CysL — декларативный язык для онтологий, LISP — функциональный язык общего назначения]
 - [CysL использует микротеоии для строгой типизации, LISP — S-выражения и макросы для гибкости]
 - [CysL специфичен для системы Cys, LISP универсален и применяется в широком спектре задач]
 - [CysL гарантирует семантическую связанность, LISP — динамическую генерацию кода]
 - }

⇒ *рекомендации по выбору**:

- { • [СусL рекомендуется для задач, требующих формализации знаний в рамках онтологий и логического вывода с учетом контекста]
- [СусL подходит для проектов, связанных с обработкой естественного языка и сложными логическими выводами]
- [LISP целесообразно использовать для разработки алгоритмов искусственного интеллекта, прототипирования и метапрограммирования]
- [LISP оптимален для задач, где важна гибкость, динамическое управление памятью и поддержка рекурсии]
- }

2 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ИНСТРУМЕНТОВ ДЛЯ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ: NELL И XLORE

NELL

- $\equiv$ [Автоматизированная система непрерывного извлечения знаний из интернета, разработанная Карнеги-Меллонским университетом]
- $\equiv$ [Система извлечения знаний, разработанная в Питтсбурге]
- $\in$ *инструмент для построения графов знаний*
 - $\equiv$ [система извлечения знаний для построения графов]
- $\in$ *семантическое программное обеспечение*
- $\Rightarrow$ *разработчик**:
Карнеги-Меллонский университет
- $\Rightarrow$ *год создания**:
2010
- $\Rightarrow$ *назначение**:
[Автоматическое пополнение баз знаний через извлечение триплетов (сущность, отношение, сущность) из текстов веб-страниц]
- $\Rightarrow$ *функциональные возможности**:
 - {
 - [постоянное извлечение знаний]
 - $\equiv$ [работа 24/7 и извлечение информации в виде сущностей и их взаимосвязей]
 - $\Rightarrow$ *пример использования**:
[Анализ текстовых данных из интернета]
 - [создание онтологий]
 - $\equiv$ [преобразование извлеченной информации в структурированный формат (например, RDF)]
 - $\Rightarrow$ *пример использования**:
[Введение новой терминологии и правил формирования заголовков в библиографическом описании]
 - [масштабируемость и долгосрочная адаптация]
 - $\equiv$ [улучшение точности извлечения информации со временем, благодаря адаптации к новым данным и расширения своих знаний без перезапуска]
 - $\Rightarrow$ *пример использования**:
[Разработка методов компактного табличного представления функций, позволяющих эффективно обрабатывать данные в долгосрочной перспективе]
 - [семантическая обработка]
 - $\equiv$ [фокусировка на понимании контекста и семантики текста, что способствует выявлению сложных взаимосвязей между сущностями]
 - $\Rightarrow$ *пример использования**:
[Анализ контекста в задачах самоконтроля качества подготовки к занятиям, где важно понимание смысла вопросов и ответов]
- $\Rightarrow$ *поддерживаемые языки**:
 - {
 - *RDF*
 - *Turtle*

```

}
⇒ преимущества*:
{• [универсальность и модульность]
  ⇒ уточнение*:
    [Позволяет адаптировать функции под различные задачи, что делает
    её гибкой для применения в разных областях.]
  • [высокая эффективность]
  • [непрерывное обучение]
  ⇒ уточнение*:
    [Имитирует человеческий стиль обучения, позволяя системе посте-
    пенно накапливать знания и улучшать понимание языка через кон-
    текст.]
}
⇒ недостатки*:
{• [энергозависимость]
  • [Сложность реализации]
  ⇒ уточнение*:
    [Для эффективной работы требуется мощная инфраструктура и ал-
    горитмы, способные обрабатывать большие объёмы данных в реаль-
    ном времени.]
}
⇒ библиографические источники*:
{• [NELL: The Computer that Learns // Carnegie Mellon
  University. – Электронный ресурс. Дата обращения: 26.05.2025.
  https://www.cmu.edu/homepage/computing/2010/fall/nell-computer-that-
  learns.shtml]
  • [Never-Ending Language Learning (NELL) // Wikipedia. – Электронный ресурс.
  Дата обращения: 26.05.2025. https://en.wikipedia.org/wiki/NELL]
  • [RTW Resources // Carnegie Mellon University. – Электронный ресурс. Дата
  обращения: 26.05.2025. http://rtw.ml.cmu.edu/rtw/resources]
  • [NELL in Ontology Development // IBM Journal of Research and Development.
  – 2012. – Т. 56, № 3–4. – С. 12:1–12:10. – DOI: 10.1147/JRD.2012.2185930.]
  • [Never-Ending Learning / Carnegie Mellon University School of Computer
  Science. – Pittsburgh, PA, 2014. – 14 н. (Технический отчет). Доступ через:
  https://www.cs.cmu.edu/~tom/pubs/NELL_aaai15.pdf]
}

```

XLore

```

:= [Англо-китайский билингвальный граф знаний на основе Wikipedia и Baidu Baike.]
∈ крупномасштабный знаниевый граф
∈ семантическое программное обеспечение
⇒ разработчик*:
  Университет Цинхуа
⇒ год создания*:
  2013
⇒ назначение*:
  [Инструмент предназначен для интеграции и структурирования междязыковых зна-
  ний, предоставляя единое пространство для работы с англо-китайскими данными]
⇒ архитектурные особенности*:
{• [многоязычная интеграция]

```

- [поддержка API]
 - [использование Python и JavaScript]
- }
⇒ *функциональные возможности**:
- {• [поиск сущностей и концепций]
 - ⇒ *уточнение**:
 [Позволяет находить сущности, концепции и свойства на основе ключевых слов через API]
 - [межъязыковые связи]
 - [обработка естественного языка]
 - [автоматизация извлечения знаний]
 - ⇒ *уточнение**:
 [Платформа использует методы моделирования и извлечения знаний для автоматического построения графа из неструктурированных и полуструктурированных источников]
- }
⇒ *преимущества**:
- {• [комплексная схема (онтология)]
 - ⇒ *уточнение**:
 [Используется тщательно спроектированная схема XLORE Ontology, которая эффективно организует и управляет сущностями из разнородных источников, обеспечивая структурированное представление знаний]
 - [многоязычная интеграция]
 - ⇒ *уточнение**:
 [Объединяет данные на китайском и английском языках, создавая сбалансированный кросс-лингвальный граф знаний с богатыми межъязыковыми связями, что полезно для мультязыковых приложений]
 - [масштабируемость и объем данных]
 - ⇒ *уточнение**:
 [Включает в себя более 50 миллионов сущностей и 500 миллионов триплетов, а также предоставляет API с ежегодным доступом более 20 миллионов запросов]
 - [автоматизированное построение знаний]
- }
⇒ *недостатки**:
- {• [зависимость от ручного проектирования]
 - ⇒ *пояснение**:
 [Некоторые методы построения графа требуют ручного создания шаблонов связей между сущностями, что ограничивает автоматизацию и увеличивает трудозатраты]
 - [ограниченная адаптация к большим данным]
 - ⇒ *пояснение**:
 [Методы, использованные в XLogе, могут сталкиваться с трудностями при обработке очень крупных наборов данных]
 - [структурные ограничения]
 - ⇒ *пояснение**:

[Граф может содержать недостаточно гибкие связи между сущностями, что затрудняет их использование в сложных аналитических задачах]

- ⇒ }
библиографические источники*:
- {• [XLORE: Кросс-лингвистический граф знаний // XLORE. – Электронный ресурс. Дата обращения: 26.05.2025. <https://xlore.cn/AboutUs>]
 - [Тан, Цзя. Интеллектуальные системы будущего: Wu Dao – крупнейшая в мире модель ИИ / Цзя Тан // Tsinghua University. – Электронный ресурс – 2021. Дата обращения: 26.05.2025. <https://www.tsinghua.edu.cn/en/info/1418/10451.htm>]
 - [ACM Digital Library // DOI: 10.1145/3660521. – Электронный ресурс. Дата обращения: 26.05.2025. <https://dl.acm.org/doi/full/10.1145/3660521>]
 - [Научные данные // SciDB. – Электронный ресурс. Дата обращения: 26.05.2025. <https://www.scidb.cn/en/detail?dataSetId=760439203967270912>]
 - [XLORE: Крупномасштабный кросс-лингвистический граф знаний / Цзя Тан [и др.] // Proceedings of the International Semantic Web Conference (ISWC 2013). – CEUR-WS, 2013. – Электронный ресурс. Дата обращения: 26.05.2025. https://ceur-ws.org/Vol-1035/iswc2013_demo_31.pdf]
- ⇒ }

Сравнение редакторов онтологий *Protégé* и *TopBraid Composer*

- = { • *NELL*
• *XLoer*
}
- ⇒ сходства*:
- {• [Предназначены для извлечения информации из неструктурированных источников]
 - [Обеспечивают семантическую обработку данных для выявления сложных взаимосвязей между сущностями]
 - [Используются в научных и промышленных проектах для структурирования знаний]
 - [Позволяют масштабировать данные и адаптироваться к новым источникам информации]
- ⇒ }
- ⇒ различия*:
- {• [NELL фокусируется на непрерывном извлечении триплетов (сущность, отношение, сущность) из интернета, в то время как XLoer специализируется на межязыковой интеграции англо-китайских данных]
 - [NELL требует мощной инфраструктуры для обработки больших данных в реальном времени, XLoer сталкивается с ограничениями при обработке очень крупных наборов из-за зависимости от ручного проектирования]
 - [NELL ориентирован на автоматическое пополнение баз знаний без перезапуска, XLoer требует частичной ручной корректировки для обеспечения гибкости связей.]
- ⇒ }
- ⇒ рекомендации по выбору*:
- {• [NELL рекомендуется для проектов, требующих непрерывного извлечения знаний из интернета и долгосрочного обучения, например, в области анализа текстовых данных или создания баз знаний с контекстным пониманием.]

- [XLogic целесообразно использовать для задач, связанных с мультиязыковыми приложениями, такими как переводы, межъязыковые поисковые системы или интеграция данных из англоязычных и китайских источников.]
- [NELL подходит для научных исследований, где важна автоматизация извлечения и обновления знаний, XLogic — для корпоративных решений, требующих структурированного представления межъязыковых данных.]
- [Если требуется высокая масштабируемость и обработка больших объемов текста в реальном времени, предпочтение стоит отдать NELL. Для задач с акцентом на межъязыковую совместимость и API-интеграцию лучше выбрать XLogic.]

}

ЗАКЛЮЧЕНИЕ

В ходе проведённого исследования был осуществлён сравнительный анализ современных технологий разработки интеллектуальных систем: языков описания онтологий и функциональных языков программирования, а также инструментов для непрерывного извлечения знаний из интернета и англо-китайского билингвального графа знаний.

В первом разделе работы были подробно рассмотрены языки CysL (язык описания онтологий) и LISP (функциональный язык программирования). Проанализированы их синтаксические и семантические особенности, выявлены общие черты и различия. Результаты анализа были формализованы с помощью SCn-кода.

Во втором разделе исследованы современные инструменты для непрерывного извлечения знаний из интернета NEll и англо-китайский билингвальный граф знаний Xlore. Проведен анализ их функциональных возможностей, а также особенностей архитектуры. Информация о каждом инструменте также была формализована на SCn-коде.

Результаты работы могут быть использованы при проектировании гибридных интеллектуальных систем, сочетающих возможности логического вывода и онтологического моделирования, а также при выборе и обосновании технологических решений для конкретных прикладных задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Lenat, Douglas B. Building Large Knowledge-Based Systems / Douglas B. Lenat, R. V. Guha. — Addison-Wesley, 1990.

[2] Lenat, Douglas B. The Dimensions of Context Space / Douglas B. Lenat, R. V. Guha. — Stanford, CA: Stanford University, 1989. — 152 pages.

[3] Matuszek, C. Using cyc to enable natural language interaction with a commercial game / C. Matuszek [et al.] // Proceedings of the 20th International Florida Artificial Intelligence Research Society Conference (FLAIRS-20). — Menlo Park, CA: AAAI Press, 2007. — P. 456–461.

[4] CycL. — <https://en.wikipedia.org/wiki/CycL>. — 2025. — Дата доступа: 24.05.2025.

[5] Cyc documentation. — <https://www.cyc.com>. — 2025. — Дата доступа: 25.05.2025.

[6] Lisp | artificial intelligence, machine learning & programming. — <https://www.britannica.com/technology/LISP-computer-language>. — 2025. — Дата доступа: 25.05.2025.

[7] Лисп. — <https://ru.wikipedia.org/wiki/Лисп>. — 2025. — Дата доступа: 25.05.2025.

[8] TechTarget,. What is the lisp (list processing) programming language? — <https://www.techtarget.com/whatis/definition/LISP-list-processing>. — 2022. — Дата доступа: 25.05.2025.

[9] McCarthy, John. Введение в программирование систем искусственного интеллекта: Tech. rep. / John McCarthy: 1958.

[10] Хабр,. Введение в язык Лисп и функциональное программирование. — <https://habr.com/ru/articles/764594/>. — 2023. — Дата доступа: 25.05.2025.

[11] University, Carnegie Mellon. Nell: The computer that learns. — <https://www.cmu.edu/homepage/computing/2010/fall/nell-computer-that-learns.shtml>. — 2025. — Дата доступа: 26.05.2025.

[12] Never-ending language learning (nell). — <https://en.wikipedia.org/wiki/NELL>. — 2025. — Дата доступа: 26.05.2025.

[13] University, Carnegie Mellon. Rtw resources. — <http://rtw.ml.cmu.edu/rtw/resources>. — 2025. — Дата доступа: 26.05.2025.

[14] in Ontology Development, NELL. Nell in ontology development / NELL in Ontology Development // IBM Journal of Research and Development. — 2012. — Vol. 56, № 3–4. — P. 12:1–12:10.

[15] of Computer Science, Carnegie Mellon University School. Never-ending learning: Tech. rep. / Carnegie Mellon University School of Computer Science. — Pittsburgh, PA: Carnegie Mellon University, 2014.

[16] Xlore: Кросс-лингвистический граф знаний. — <https://xlore.cn/AboutUs>. — 2025. — Дата доступа: 26.05.2025.

[17] Тан, Цзя. Интеллектуальные системы будущего: Wu dao – крупнейшая в мире модель ИИ. — <https://www.tsinghua.edu.cn/en/info/1418/10451.htm>. — 2021. — Дата доступа: 26.05.2025.

[18] Acm digital library. — <https://dl.acm.org/doi/full/10.1145/3660521>. — 2025. — Дата доступа: 26.05.2025.

[19] Научные данные. — <https://www.scidb.cn/en/detail?dataSetId=760439203967270912>. — 2025. — Дата доступа: 26.05.2025.

[20] Тан, Цзя. Xlore: Крупномасштабный кросс-лингвистический граф знаний / Цзя Тан [et al.] // Proceedings of the International Semantic Web Conference (ISWC 2013). — CEUR-WS, 2013. — Дата доступа: 26.05.2025.